

固化版本 + 对比实验报告

1D-AM Branch-and-Bound · 单机 oracle + 并行机 B&B · 2026-06-20

本文把当前 oracle / 并行 B&B 固化为一个基准版本,并给出对比实验:单机正确性回归、并行机 B&B(diving 规则修正 + 消融)、规模扩展(n=16-20)。所有运行用 g++ -O2 在统一环境下完成。

0. 固化的版本(冻结配置)

单机 oracle(`oracle.cpp` → `oracle`)

项	默认值
分支	Type I / Type II($j > oMax$ 递增对称破除)
下界	closed(精确) + 开口批次 floor + max(LB_par, LB_pos)
interch / pos	1 / 1(交换支配、LB_pos 均开)
qfrac	0.5(LB_pos 浅层门控)
N_MAX / N_MIN	200000 / 50000
FRONT_CAP	25,000,000(支配前沿键数上限)

构建:`cd new_oracle && g++ -std=c++17 -O2 -o oracle oracle.cpp`

并行机 B&B(`ParallelBranchBound.cpp` → `pbb`)

项	默认值
方案	方案 A:零件→机器指派分支,叶子用单机 Φ 精确求值
Φ	<code>new_oracle/phi_oracle.h::solvePhi</code> (强 oracle,~5× 旧版)
diving	depth-first(深度优先下潜:最深优先,平手取 min ℓ ;DIVE=load 退回旧规则)
LS	1(move 邻域局部搜索初始上界,开)
WARM / FREEPAR	0 / 0(均关;FREEPAR 已证被 singleTard 压制)
N_max / N_min	200000 / 50000

构建:cd src && g++ -std=c++17 -O2 -o pbb main.cpp ParallelBranchBound.cpp BranchBound.cpp
InstanceData.cpp

1. 单机正确性回归(new_oracle on data/)

全部 11 个仓库算例:目标值与历史基准逐一一致、均可证最优(gap 0%)。含上一版翻车的
10part_2 / 10part_3。

实例	n	目标值(TT)	proven	popped	时间
5part	5	40.5691	是	26	0.000s
10part	10	116.30	是	6,192	0.010s
10part_2	10	81.7384	是	3,150	0.005s
10part_3	10	136.74	是	4,005	0.006s
10parts_4	10	75.7821	是	4,082	0.007s
11part	11	110.607	是	19,264	0.032s
12part	12	139.084	是	53,626	0.115s
13part	13	168.723	是	201,741	0.409s
14part	14	156.522	是	388,614	0.747s
15part	15	139.061	是	577,447	1.092s
15part_2-5	15	118.238	是	834,240	1.577s

结论:固化版本通过全量正确性自检。

2. 并行机 B&B:diving 规则修正 + 消融(M=2 / 3 / 4)

本轮把搜索内的 diving 规则从旧的 min- ℓ (均衡优先)改成 depth-first(深度优先下潜:最深节点优先,平手取最均衡)。两者最优值完全一致,只比效率与内存。peak = 活动节点表的峰值规模(内存代理指标)。

2.1 Diving 规则对比(头条)

实例	M	min- ℓ 节点	min- ℓ peak	depth 节点	depth peak
13part	2	5,945	1,683	5,065	465
13part	3	97,990	30,469	5,119	366
13part	4	25,277	6,926	1,293	23
14part	2	9,781	1,662	6,397	321

14part	3	15,378	4,777	6,399	398
15part	2	22,305	6,269	9,865	1,108
15part	3	93,975	14,905	3,399	23
16part	3	184,275 †	122,602	52,173	4,413
18part	3	65,022 †	129,940	29,255	33

† = 30-35s 内未证完;depth 在这两例分别 4.2s / 1.08s 证到最优。

- 峰值前沿坍塌 3-4 个数量级(15part M=3:14,905→23;18part M=3:129,940→33)。这才是 dfsMode 内存阀门本该有的行为——旧 min-ℓ 按负载层级广度扫,前沿只涨不收,正是 n=19/20 OOM 的根因。
- 节点数在 M≥3 大幅下降(15part M=3:94k→3.4k;13part M=3:98k→5.1k)。
- 大算例上 min-ℓ 证不完而 depth 几秒证完;16part M=3 里 min-ℓ 跑满 30s 增量解还卡在 25.27,depth 很快潜到真最优 20.68。

2.2 局部搜索(LS)边际效应(depth 默认下)

换上 depth 下潜后,LS 的边际收益明显变小(下潜本身就快速拿到均衡好叶子)。Default(LS 开) vs LS off,生成节点:

实例	M	Default(LS 开)	LS off
13part	2	5,065	5,215
13part	3	5,119	6,127
13part	4	1,293	4,457
14part	2	6,397	5,877
14part	3	6,399	2,445
15part	2	9,865	9,881
15part	3	3,399	8,910
15part	4	0	1,546

- LS 现在是小幅、依实例——15part M=3/M=4 仍有用,14part M=3 反而略增。下潜规则修对后,LS 之前干的“快速收紧增量解”大部分被下潜本身吃掉,两者部分冗余。默认仍开(便宜、永不抬高最优)。

2.3 FREEPAR 仍是 no-op

新默认下复测:15part M=2/M=3 开关 FREEPAR 节点数逐位相同(被 singleTard 结构性压制,与节点顺序无关)。默认关。

3. 规模扩展(n=16-20,单机 oracle)

合成 16-19 件(论文 TF=RDD=0.6、20×20 平台、固定种子)+ 仓库 20 件算例。本表时限为沙箱短时限(32-35s,仅作规模演示);本机用 600-3600s 可证更多。gap = (obj-lb)/obj。

实例	n	obj(最好解)	状态	lb(下界)	gap	时间
16part	16	304.217	可证最优	304.217	0.00%	12.4s
17part	17	206.31	可证最优	206.31	0.00%	15.8s
18part	18	232.734	增量解(超时)	133.637	42.6%	35s
19part	19	352.096	增量解(超时)	124.476	64.7%	35s
20part_3-S	20	224.652	增量解(超时)	62.18	72.3%	32s
20part_4-S	20	240.99	增量解(超时)	67.98	71.8%	32s

- n≤17:十几秒内证到最优(16part 12.4s、17part 15.8s)。
- n=18-20:短时限下给强上界,但 gap 大——瓶颈在下界侧(廉价界天花板),不是上界。

4. 一页结论

维度	状态
单机正确性	全量 11 算例与基准一致、全部可证最优
单机规模	n≤17 数十秒证完;n=18-20 给强增量解(界受限)
并行 diving	depth-first 取代 min-ℓ:峰值前沿降 3-4 数量级,M≥3 节点降 1-2 数量级,大算例从证不完→几秒证完
并行 M=2	LB-limited,初始上界已近最优;diving 改进主要体现在内存/节点常数
并行 M≥3	depth 下潜主导收益;LS 退为小幅依实例的廉价精修
FREEPAR	零效果(已证死路),默认关

固化结论:depth-first diving 已扶正为默认;该配置在正确性、效率与内存上均为各规模的稳态最优,作为后续工作的基准版本。